

Who Wrote It? Identifying LLM Families from Their Responses

CMPSC 448 Midterm Project, Fall 2026

Team leader and sole member: Wyatt Allison (individual project)

Repository: <https://github.com/WyattA-25/llm-fingerprint>

Abstract. Can we tell which family of large language model (LLM) wrote a response? Using 15,999 responses from eight families drawn from the 2025 Chatbot Arena release, we train a character-level CNN and a word-level BiLSTM, plus TF-IDF and style-feature baselines. With model names masked and no prompt shared between training and test, the CNN reaches 73.6% accuracy and TF-IDF 74.8% (chance: 12.5%). The prompt alone carries no signal (12.7 to 13.5%). Fingerprints survive a change of task domain at a cost of up to 15.7 points. Ablations show the CNN relies mostly on formatting, yet a model retrained with all formatting removed still reaches about 52%: the fingerprint is both structural and lexical.

1. Problem Definition and Dataset Curation

Task. Each example is a triple (LLM_name, LLM_input, LLM_output). We map each model to one of eight families and treat identification as 8-way classification on balanced classes (chance 12.5%), reporting accuracy and macro-F1 on a held-out test set.

Source. We use `lmarena-ai/arena-human-preference-140k` [2], 135,634 Chatbot Arena "battles" [1] collected from April to July 2025. In each battle a real user writes a prompt and two anonymous, randomly assigned models answer it. We chose it over generating responses through APIs (costly, rate-limited on free tiers, and no free Claude tier) and over older Arena releases (2023-era models). It provides modern models, real prompts, two answers per prompt (a built-in control for RQ2), and category tags we use as domain labels (RQ3). The cost is that we control neither system prompts nor sampling settings, and each family mixes several model versions. The dataset is released under CC-BY-4.0, which permits redistribution with attribution, so the repository includes the filtered file. All prompts were publicly released by LMArena; we use no private data.

Construction (`prepare_data.py`). We keep English, single-turn battles; treat each side as one example; keep prompts of 10 to 4,000 characters and responses of 50 to 8,000; map model names to families by regular expression (Table 1), excluding other vendors and Google's open Gemma models (a different tier from Gemini); remove duplicates; and randomly sample 2,000 responses per family. One near-empty response was later dropped, leaving 15,999. Domains come from the dataset's annotations (code, math, creative writing, otherwise general); every family has nearly the same mix (about 29% code, 8% creative writing, 57% general, 6% math), so topic cannot act as a shortcut.

Table 1. Families and model versions, with responses available before balancing.

Family	Models	Model versions	Eligible	Kept
Claude	8	Opus 4, Sonnet 4, 3.7 Sonnet, 3.5 Sonnet, 3.5 Haiku (incl. thinking variants)	23,807	2,000
GPT	8	o3, o3-mini, o4-mini, GPT-4.1, GPT-4.1-mini, ChatGPT-4o, GPT-4o, GPT-4o-mini	18,608	2,000
Gemini	8	2.5 Pro, 2.5 Flash, 2.5 Flash-Lite (incl. previews), 2.0 Flash	14,612	2,000

Family	Models	Model versions	Eligible	Kept
Qwen	6	Qwen3-235B (thinking / no-thinking / 2507), Qwen3-30B, Qwen-Max, QwQ-32B	12,557	2,000
Llama	4	Llama 4 Maverick (2 versions), Llama 4 Scout, Llama 3.3 70B	7,927	2,000
Mistral	4	Mistral Medium, Mistral Small 3.1 and 2506, Magistral Medium	6,857	2,000
Grok	4	Grok 3, Grok 3 mini (beta, high), Grok 4	5,671	2,000
DeepSeek	2	DeepSeek-R1-0528, DeepSeek-V3-0324	5,369	2,000

Leakage controls (src/preprocess.py). Each targets a way the classifier could succeed for the wrong reason. (1) **Name masking**: vendor and model names (OpenAI, GPT-*, Anthropic, Claude, Gemini, Llama, Qwen, DeepSeek, Mistral, Grok, xAI, and variants) become [MODEL] in prompts and responses, so "I'm Claude" cannot give the answer away. A spot check of 25 masked responses found one false positive (the English verb "grok"). Masking is imperfect: Grok mentions itself most, so the mask token still carries some signal; RQ4 measures how much. (2) **Reasoning traces**: visible <think> blocks are stripped (only one existed). (3) **Grouped splits**: each prompt can appear twice, once per battle side, so we split 70/15/15 by prompt and assert no prompt crosses splits (11,198 / 2,389 / 2,412 rows). (4) **Matched pairs**: 2,301 rows are prompts answered by two different families, 371 of them in the test set, used as a control in RQ2. Table 2 lists per-family flags used in the analysis.

Table 2. Per-family flags. "Names own family" counts any mention of the name, so it overstates self-identification.

Family	Reasoning variant	Refusal	Names own family	Median words
Claude	30.6%	3.2%	2.8%	180
DeepSeek	49.4%	1.0%	0.8%	372.5
GPT	49.8%	1.2%	4.7%	361
Gemini	11.3%	1.1%	1.9%	524
Grok	57.8%	0.7%	6.9%	593
Llama	0.0%	0.3%	1.5%	441.5
Mistral	11.8%	1.1%	1.1%	349
Qwen	50.4%	1.6%	0.8%	454

Refusals and self-mentions are rare (at most 3.2% and 6.9%), so neither can explain high accuracy. The reasoning-variant share ranges from 0% (Llama) to 58% (Grok), a confound we revisit in Section 4.3.

2. Models and Training

CNN. The CNN reads characters, because the cues we expected (markdown, line breaks, bullet symbols, curly quotes, dashes) are what word tokenizers discard. Input is the first 2,000 characters over a 303-symbol vocabulary. Following Kim [3] and Zhang et al. [4]: a 64-dimensional embedding; parallel convolutions of widths 3, 5, and 7 (128 filters each) with batch norm and ReLU; max-pooling; a second width-5 convolution that widens the receptive field to about 20 characters (enough for patterns like a newline followed by a bullet and bold text); global max- and mean-pooling ("did a pattern occur?" and "how often?"); dropout 0.3; and a linear layer (888,008 parameters).

BiLSTM. The RNN reads lowercased words, keeping punctuation and line breaks as tokens, over a 30,003-token vocabulary with a 400-token limit. A 128-dimensional embedding feeds a one-layer bidirectional LSTM [5] (128 units per direction, packed sequences), then additive attention pooling [6], dropout 0.3, and a linear layer (4,123,145 parameters, mostly in the embedding). Reading in both directions suits document-level habits such as opening with praise and closing with an offer to help.

Baselines. TF-IDF uses word 1 to 2-grams and case-sensitive character 2 to 5-grams with logistic regression, over the whole response. The stylometric model sees no words, only 27 numbers per response (length, lexical diversity, markdown density, punctuation and typography rates, emoji, and conversational habits such as opening with "Great question"), fed to gradient-boosted trees.

Table 3. Training configuration.

Setting	CNN and BiLSTM
Optimizer and loss	AdamW (lr 1e-3, weight decay 1e-4), cross-entropy
Batches	64 examples, bucketed by length to minimize padding
Schedule and stopping	Halve lr when validation macro-F1 stalls; stop after 4 epochs without improvement (max 20); restore best checkpoint
Regularization	Dropout 0.3, gradient clipping at 1.0, batch norm (CNN)
Seeds and hardware	3 seeds (0, 1, 2); NVIDIA GPU, PyTorch 2.5.1; about 24 s (CNN) and 10 s (BiLSTM) per epoch

The validation set is used only for early stopping and scheduling. All models, including the baselines, learn from the same training split. For RQ2's input + output condition, the prompt may use at most 25% of the length budget, followed by a separator and the response. In every run, training loss kept falling while validation F1 leveled off (around 0.75 for the CNN and 0.60 for the BiLSTM), so early stopping, not the epoch limit, ended training. After the laptop's sleep and restarts cost hours of an early run, we cached every finished run under a hash of its configuration, seed, and data, so the pipeline resumes after interruption. As a check, the CNN retrained from scratch in RQ2 matched its RQ1 accuracy exactly (73.56%).

3. Results

3.1 RQ1: Can we identify which LLM generated a response?

Table 4. RQ1 test results, output only (n = 2,412). Neural models: mean $\pm$ standard deviation over 3 seeds.

Model	Accuracy (%)	Macro-F1 (%)
TF-IDF + logistic regression	74.8	74.4
CNN (required)	73.6 $\pm$ 0.7	73.5 $\pm$ 0.7
Stylometric features only	61.2	60.6
BiLSTM (required)	60.7 $\pm$ 1.2	59.8 $\pm$ 1.3
Chance	12.5	12.5

Yes, by a wide margin: the CNN is right 5.9 times as often as chance. The ranking was not what we expected: TF-IDF is best, the CNN 1.2 points behind, and the BiLSTM 14.1 behind, level with the word-free stylometric model. Three reasons are likely. 11,198 examples is little data for models trained from scratch;

character n-grams already capture most formatting cues the CNN must learn; and TF-IDF sees the whole response while the neural models truncate, which matters because longer responses are easier (TF-IDF accuracy rises from 65% in the shortest quarter to 82% in the longest). The BiLSTM also loses case through lowercasing and overfits its large embedding table. For comparison, Sun et al. [7] reach 97.1% on five LLMs by fine-tuning large pretrained embedding models on answers to identical prompts; a lower ceiling is expected with eight classes, uncontrolled prompts, mixed versions, and small models trained from scratch.

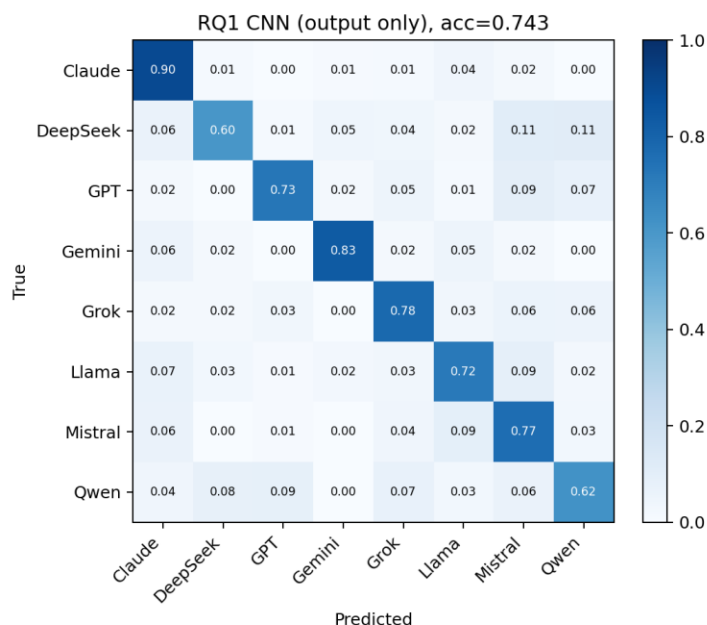


Figure 1. Row-normalized confusion matrix of the best CNN seed.

Claude (91% TF-IDF recall) and Gemini (90%) are easiest; Mistral (57%), DeepSeek, and Qwen (both 64%) are hardest. The main confusions are DeepSeek and Mistral (13% and 12% of each family's responses go to the other) and Qwen and GPT (about 10%), pairs that share a heavy markdown style (Section 4.2). The CNN beats TF-IDF on Mistral (74% vs. 57%) but trails it on GPT, so the two make partly different errors.

3.2 RQ2: Does the user's prompt help identify the LLM?

Table 5. RQ2 accuracy (%) on the full test set ($n = 2,412$) and on matched-prompt pairs ($n = 371$).

Model	Input only	Output only	Input + output	Output only, matched pairs
TF-IDF	13.5	74.8	74.1	72.5
CNN	12.8 ± 0.2	73.6 ± 0.7	72.9 ± 0.4	74.4 ± 0.2
BiLSTM	12.7 ± 0.5	60.7 ± 1.2	60.2 ± 1.7	58.9 ± 2.6

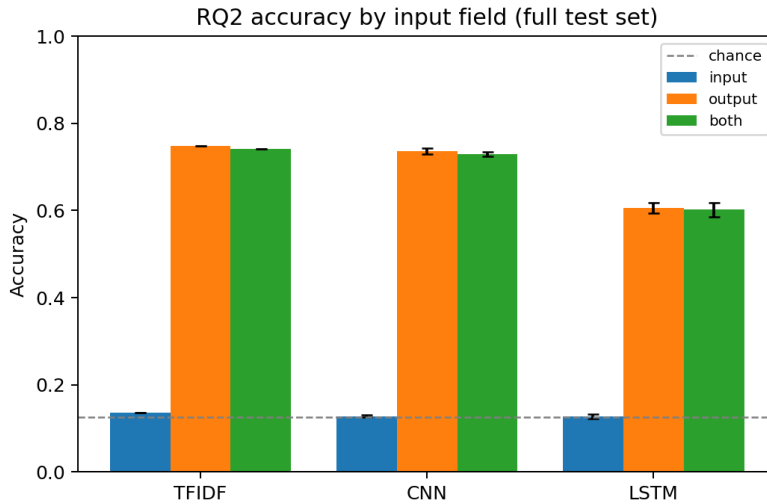


Figure 2. RQ2 accuracy by input field (error bars: standard deviation over seeds).

(i) The LLM cannot be predicted from the prompt. Input-only accuracy is 12.7 to 13.5%, not significantly above chance (best model vs. 12.5%: $p \approx 0.13$). Its most heavily weighted n-grams are unrelated topic words ("summer" for Claude, "dog" for Qwen), the signature of fitting noise.

(ii) Adding the input does not help. For all three models, input + output scores slightly below output only (−0.6, −0.6, −0.5 points): the prompt adds noise and uses up length budget.

(iii) Why input-only can work, and why it doesn't here. We predicted a timing confound: models are live on Arena at different times, so their prompts would come from different months. That prediction was wrong, because Arena assigns models to battles at random, so a prompt does not depend on which model answers it. The data also spans only about three months, with nearly every family present throughout. A classifier given only the timestamp does reach 19.4% (mostly by recognizing the late-arriving Mistral and DeepSeek), so a time signal exists, but prompt content barely carries it. Where users choose which assistant to ask (shared ChatGPT conversations, logs from one product), input-only classification would likely succeed because people ask different assistants different things, but that would describe data collection, not the models. Input-only accuracy is best read as a leakage test for the dataset.

Matched pairs confirm it. On the 371 test responses whose prompt was also answered by another family, the input is identical within each pair, so an input-only model must give both the same label (verified: 100% of pairs) and can be right on at most one. Output-only accuracy there (72.5% TF-IDF, 74.4% CNN) matches the full test set: the fingerprint is in how a model answers, not what it was asked.

4. In-depth Analyses

4.1 RQ3: Do fingerprints generalize across domains?

For each target domain, 30% of its prompts form a test set. We train on an equal number of examples either from the same domain (in-domain) or only from the other three (cross-domain) and test both on the same rows. Matching training size ensures a drop reflects domain shift, not less data. Neural results average 3 seeds.

Table 6. RQ3 accuracy (%), in-domain vs. cross-domain with size-matched training. Neural models: mean $\pm$ std over 3 seeds; TF-IDF is deterministic.

Test domain	Train / test	TF-IDF in / cross	CNN in / cross	BiLSTM in / cross
code	3,307 / 1,419	70.4 / 65.5	67.7 ± 2.0 / 60.3 ± 1.6	56.1 ± 1.7 / 46.6 ± 0.2
general	6,344 / 2,713	74.8 / 70.4	73.4 ± 1.2 / 72.0 ± 0.4	57.8 ± 0.4 / 55.2 ± 0.2
creative_writing	876 / 394	58.4 / 51.0	52.5 ± 1.6 / 45.6 ± 1.1	34.6 ± 2.5 / 32.3 ± 1.1
math	665 / 281	62.3 / 50.2	60.5 ± 1.8 / 44.8 ± 3.1	43.8 ± 3.0 / 31.9 ± 2.1

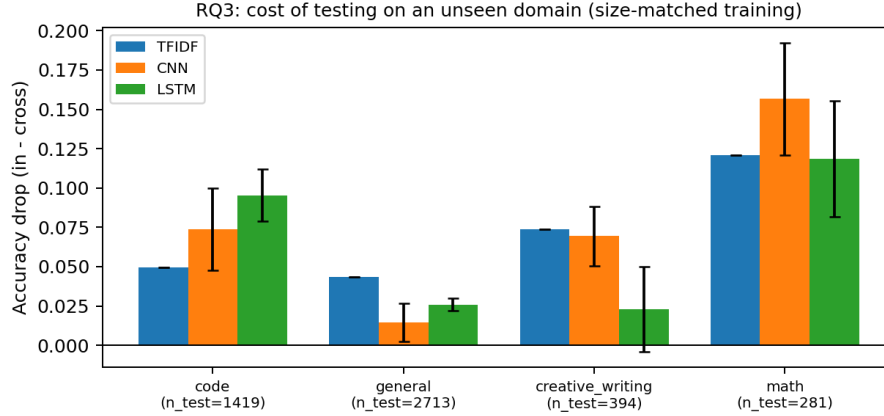


Figure 3. Accuracy lost when the test domain was unseen in training.

(i) Performance drops, but the fingerprint largely transfers. Accuracy falls in 12 of 12 model and domain combinations, by up to 15.7 points, but cross-domain accuracy never nears chance (lowest 31.9%), and TF-IDF and the CNN keep 74% to 98% of their in-domain accuracy. General questions transfer best (CNN -1.4), likely because "general" already mixes many styles. Math transfers worst (TF-IDF -12.1 , CNN -15.7), since math answers follow their own format (LaTeX, numbered derivations). On code the CNN loses 7.4 points and TF-IDF 4.9, a gap close to the seed-to-seed variation, so we do not claim either model transfers better.

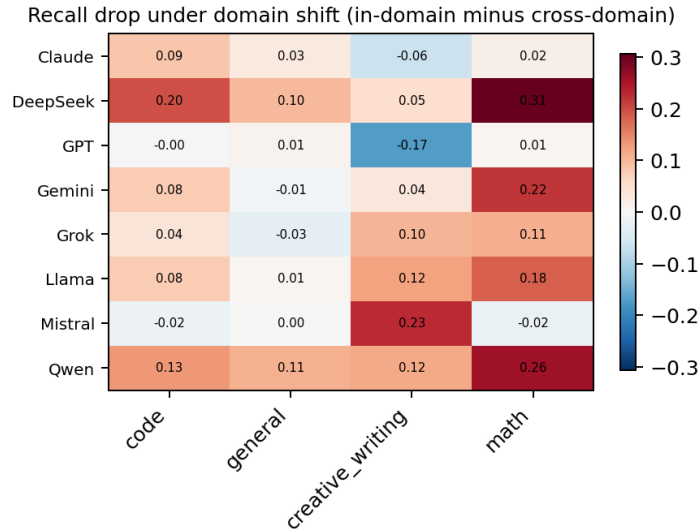


Figure 4. Per-family recall lost under domain shift, averaged over models (positive: harder in an unseen domain).

(ii) Some families travel better. GPT and Mistral transfer to code and general prompts with essentially no loss (Grok loses little), while DeepSeek and Qwen lose recall in every domain. Math hurts most families, but not Claude, GPT, or Mistral. With only about 35 to 50 test responses per family in math and creative writing, single cells carry roughly 8 points of standard error, so we read only consistent patterns. **(iii) General or task-specific?** Both: task-specific patterns alone would collapse to chance, and a purely general fingerprint would show no drop. There is a domain-independent core plus a part that depends on how each family formats a given kind of answer, largest where formats differ most (math).

4.2 RQ4: What characteristics distinguish the families?

Table 7. Measured habits that distinguish each family (stylometric means and top TF-IDF features).

Family	Distinctive measured habits
Claude	Shortest (median 180 words); least bold text; never uses curly quotes; almost never opens with praise (1%); plain "-" bullets.
GPT	Most curly quotes (3.2 per 1,000 characters) and em dashes; "•" bullets; frequent horizontal rules.
Gemini	Opens with an affirmation such as "Okay," in 41% of responses; almost never ends with an offer to help (3%); "*" bullets.
Qwen	Most horizontal rules (4.1 per response), headers, and emoji.
DeepSeek	Heaviest bold text (5.6 per 100 words) and many headers.
Mistral	Heavy bold; ends with "Would you like..." offers (38%).
Llama	Lowest lexical diversity; numbered lists with bold labels.
Grok	Longest (median 593 words); connective phrasing ("such as", "if you", "based on").

These habits explain the RQ1 confusions: DeepSeek, Mistral, and Qwen share dense bold, headers, and bullets, while Claude and Gemini each have habits no one else shares.

(i) Which signals does the classifier use? Structure alone goes far: the stylometric model, which never sees a word, reaches 61.2%. Its most important feature (Figure 5) is the curly-quote rate, followed by bold,

headers, horizontal rules, blank lines, lexical diversity, and length. Whether a model writes ' or ' is a byproduct of training data that no human reader would notice. The learned models agree: GPT's top three TF-IDF features are curly-quote characters, Gemini's are "okay ," and "*" bullets, and the BiLSTM's attention concentrates on formatting tokens (#, line breaks, quotes, dashes) rather than content words.

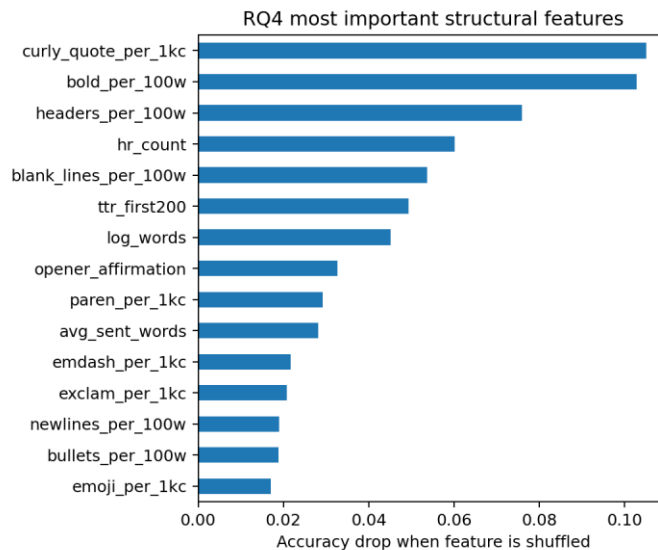


Figure 5. Permutation importance of the top 15 stylometric features.

(ii, iii) **Linguistic, structural, or both? Does removing signals hurt?** We remove one kind of signal and measure accuracy two ways: **retraining** on ablated text (is the information needed?) and **testing the clean-trained CNN** on ablated text (does the model rely on it?). CNN retraining averages 3 seeds.

Table 8. RQ4 ablation accuracy (%).

Ablation	Removes	TF-IDF retrained	CNN retrained	Clean CNN, ablated test
none	Nothing	74.8	73.6 ± 0.7	73.7
strip_markdown	Headers, bullets, bold, tables, rules	71.5	67.4 ± 0.1	52.2
flatten_whitespace	All line breaks	73.3	68.9 ± 1.7	56.4
normalize_punct	Curly quotes, dashes, emoji	72.7	73.4 ± 0.7	70.6
lowercase	Case	74.5	73.9 ± 1.1	71.9
fixed_length_600	Everything after 600 characters	64.8	71.0 ± 0.2	68.9
all_structure_removed	All of the above	53.5	52.4 ± 0.2	23.6
no_masking	Masking (names left visible)	74.9	75.1 ± 0.1	73.7

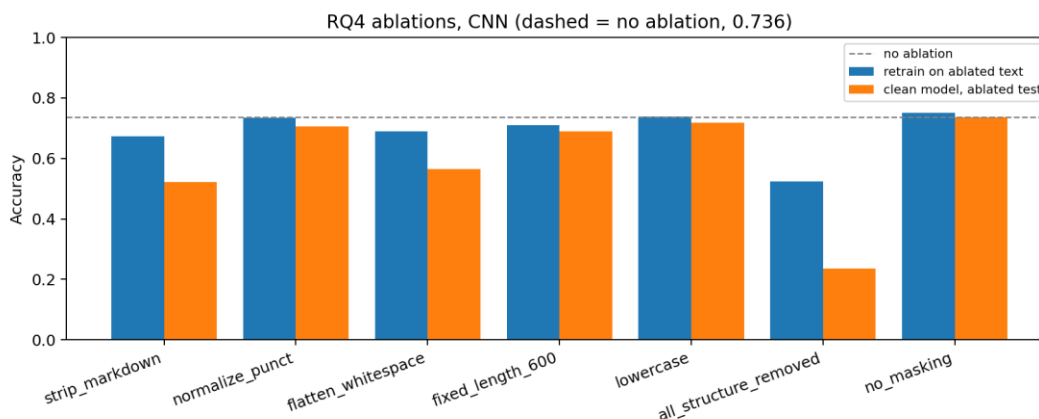


Figure 6. CNN ablations: retrained on ablated text (blue) vs. trained on clean text and tested on ablated text (orange).

The CNN relies on formatting: stripping markdown at test time drops it from 73.7% to 52.2%, and removing all structure to 23.6%. **But formatting is not required:** retrained without markdown, the CNN recovers to 67.4%, and with every structural signal removed at once, retrained models still reach 53.5% (TF-IDF) and 52.4% (CNN), about four times chance. What remains is lexical: word choice, phrasing, and how each family opens its answers. So the signals are both structural and linguistic; structure dominates what the trained models use, and words alone suffice for about half. The gap between the two columns is the main lesson: what a model relies on says little about what the task requires. Individually, lowercasing and punctuation normalization cost nothing measurable after retraining, even though curly quotes are the top stylometric feature: correlated cues compensate. Leaving names unmasked changes accuracy by only +0.1 (TF-IDF) and +1.5 (CNN) points, so self-identification explains almost none of the performance.

4.3 Families are not uniform

Accuracy varies more within some families than between them. Every Claude version scores at least 83%, suggesting a stable house style. Other families are mixtures: Magistral, Mistral's reasoning model, is recognized as Mistral only 34% of the time; Grok 3 mini scores 89% but Grok 3 preview 57%; ChatGPT-4o scores 47% but o3 93% (TF-IDF). A family fingerprint is therefore a blend of per-model fingerprints, weighted by whichever models dominate the sample. This also qualifies a finding from RQ1: reasoning variants are more identifiable (79.7% vs. 72.3%), but they are concentrated in a few families and include distinctive models (o3, o4-mini, Grok 3 mini), so we cannot attribute the gap to reasoning itself.

4.4 Limitations

Arena's system prompts and sampling settings are unknown and may differ by model. Results depend on the mix of versions within each family. The data covers three months, English only, single-turn. Domain tags are automatic and imperfect, and regex masking misses paraphrased self-references. Truncation differs by model (2,000 characters, 400 tokens, none), which favors TF-IDF on long responses. Per-family RQ3 results on math and creative writing are noisy.

5. Lessons and Experience

Operations cost more time than modeling. My setup had CPU-only PyTorch (the default Windows install), so I reinstalled the CUDA build. After that a CNN epoch took about 25 seconds. My first full overnight run still failed. The laptop went to sleep, so single epochs were logged as taking one to four hours, and it then restarted after about six hours with only RQ1 finished. I also lost time to smaller problems: running a script from the wrong folder, a pasted line that merged two lines into a syntax error, a library update that changed the datasets API, and a dependency conflict after reinstalling PyTorch. The fix that mattered most was making the pipeline resumable. Every finished training run is cached under a hash of its configuration, seed, and data, so a crash costs minutes instead of hours. I also disabled sleep while plugged in and paused Windows updates. From day one next time, I would check `torch.cuda.is_available()` before anything else, build in resumability before the first long run, and test the whole pipeline on small synthetic data first.

A strong baseline is the most honest benchmark. I expected the CNN and BiLSTM to beat TF-IDF. Instead TF-IDF was best (74.8%), the CNN close behind (73.6%), and the BiLSTM far behind (60.7%). A model given only 27 hand-crafted style numbers reached 61.2%, matching the BiLSTM. With about 11,000 training examples, character n-grams already capture most of the formatting cues the CNN has to learn from scratch. The lesson is to build simple baselines first. They show whether a neural model's extra complexity is buying anything, and the stylometric baseline told me what kind of signal existed before RQ4 confirmed it.

How data is collected decides which shortcuts exist. Before running RQ2, I predicted the prompt alone would identify the model above chance, because different models were live on Arena at different times. I was wrong: input-only accuracy was 12.7% to 13.5%, which is chance. Arena assigns models to battles at random, so a prompt does not depend on which model answers it, and my data covers only about three months. A timestamp-only model did reach 19.4%, so the time signal was real but barely present in the prompts. I learned to treat input-only accuracy as a test of the dataset rather than of the models, and that writing a prediction down before running an experiment makes a wrong answer informative.

Leakage controls are cheap, and measuring them turns worries into numbers. Grouped splits were essential, because every prompt can appear twice (once for each model in a battle). The code asserts that no prompt crosses splits. For name masking, I checked 25 random masked responses and found one false positive (the English verb "grok"). The no-masking ablation then showed masking mattered less than I feared: leaving names visible added only 1.5 points for the CNN. The matched pairs confirmed that output-only accuracy holds when the prompt is identical. Each control took little effort to add, and each turned a vague doubt into a measured answer.

The "family" label hid the most interesting result. Every Claude version scored at least 83%, but Mistral's reasoning model Magistral was recognized as Mistral only 34% of the time, and ChatGPT-4o scored 47% while o3 scored 93%. A family classifier partly learns whichever models dominate that family's sample. If I redesigned the task, I would classify individual models, or classify models first and then group them into families, and I would balance the number of responses per model within each family.

With another month, I would build a controlled dataset. I would send the same prompts to each model through their APIs, with the same system prompt, temperature, and length limit, and balance responses

per model. This addresses the biggest weakness of this project: different models answered different prompts under settings I could not see. It would show whether 74.8% is a floor or inflated by those differences, and it would make the model-level analysis above possible.

6. Reproducibility and Use of AI Tools

`prepare_data.py` builds the dataset; `src/` holds preprocessing, models, and training; `experiments/rq1.py` to `rq4.py` produce every number and figure, saved in `results/`. `python run_all.py` reproduces everything (seeded); `python run_all.py --smoke` checks the pipeline on synthetic data in minutes.

Use of AI tools. As the assignment permits, I used Claude (Anthropic) throughout this project. Claude proposed the data source and the experimental design, including the leakage controls and the controls in RQ2 and RQ3. It wrote most of the code, helped me debug environment and dependency problems, helped interpret results, and drafted this report, including the Lessons section from my account of what happened. I made the project decisions: working solo, using a free dataset, attempting all four research questions, and the report's format and length. I ran every experiment on my own machine, inspected the masking output myself, and reviewed the final text. I am responsible for everything submitted and can explain each design choice.

References

- [1] W.-L. Chiang et al. Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference. ICML, 2024.
- [2] LMArena. arena-human-preference-140k. Hugging Face dataset, 2025. <https://huggingface.co/datasets/lmarena-ai/arena-human-preference-140k>
- [3] Y. Kim. Convolutional Neural Networks for Sentence Classification. EMNLP, 2014.
- [4] X. Zhang, J. Zhao, Y. LeCun. Character-level Convolutional Networks for Text Classification. NeurIPS, 2015.
- [5] S. Hochreiter, J. Schmidhuber. Long Short-Term Memory. Neural Computation, 9(8), 1997.
- [6] D. Bahdanau, K. Cho, Y. Bengio. Neural Machine Translation by Jointly Learning to Align and Translate. ICLR, 2015.
- [7] M. Sun, Y. Yin, Z. Xu, J. Z. Kolter, Z. Liu. Idiosyncrasies in Large Language Models. ICML, 2025.